

4. Git e GitHub

4. Git e GitHub

Una settimana dopo aver corretto un bug critico nel checkout di ShopFacile, Marco lo ritrova identico, di nuovo in produzione. Ricostruendo i fatti con Ahmed capisce cosa è successo: si erano scambiati il file `checkout.js` via chat per un intervento urgente, ognuno partendo dalla propria copia locale. Ahmed ha salvato sopra la sua versione più vecchia senza saperlo, cancellando la correzione di Marco — e nessuna delle due copie porta scritto chi l'ha modificata, quando, o perché.

Git e GitHub esistono per risolvere esattamente questo problema, ma applicato al codice sorgente di un software: migliaia di file che decine di persone modificano ogni giorno, in parallelo, senza perdere traccia di chi ha cambiato cosa.

Questa è probabilmente la sezione più “pratica” del corso: gli strumenti che imparerai qui li vedrai citati in ogni standup, in ogni pull request, in ogni pipeline di CI/CD. Non serve che tu diventi uno sviluppatore, ma è fondamentale che tu capisca il vocabolario e la logica di fondo, perché è il linguaggio con cui il tuo team lavora ogni giorno.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- spiegare la differenza tra Git e GitHub;
- capire cosa sono repository, commit, branch e merge;
- capire come funziona una Pull Request e perché è il cuore della collaborazione su GitHub;
- distinguere Issue, Release e Tag;
- conoscere tre modelli di lavoro (workflow) diffusi: **Git Flow**, il modello **cactus (OneFlow)** e il **Trunk Based Development**, e capire quando si usa l'uno o l'altro;
- riconoscere a cosa serve **Maven** e il file `pom.xml` nei progetti Java, e capire perché la gestione delle dipendenze esterne è un problema comune a ogni linguaggio di programmazione.

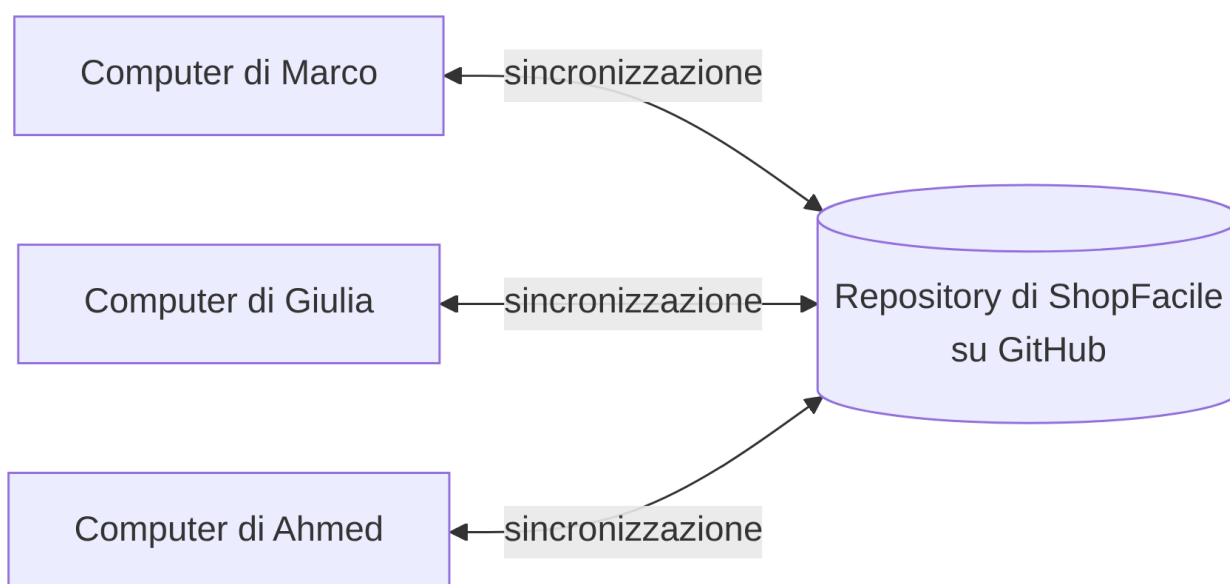
4.1 Cos'è Git

Git è un **sistema di controllo di versione** (in inglese *Version Control System*, o VCS). In parole semplici: è un programma che tiene traccia di **ogni modifica** fatta ai file di un progetto, salvando nel tempo tutta la “cronologia” di quelle modifiche.

Analogia: pensa alla cronologia delle modifiche di Google Docs o alla funzione “Traccia modifiche” di Word, ma molto più potente. Con Git non solo puoi vedere chi ha cambiato cosa e quando, ma puoi anche: - tornare indietro a una versione precedente in qualsiasi momento; - far lavorare **più persone in parallelo** sullo stesso progetto senza che si sovrascrivano il lavoro a vicenda; - “unire” automaticamente le modifiche fatte da persone diverse.

Git è stato creato nel 2005 da Linus Torvalds (lo stesso creatore di Linux) per gestire lo sviluppo del kernel Linux, un progetto con migliaia di collaboratori. Oggi è lo standard de facto per versionare codice, usato praticamente da ogni azienda software al mondo.

Un punto fondamentale: **Git è distribuito**. Questo significa che ogni persona che lavora su un progetto ha, sul proprio computer, una **copia completa** di tutta la cronologia del progetto — non solo l'ultima versione, ma *tutta la storia* fin dall'inizio. Questo è molto diverso dai vecchi sistemi centralizzati, dove esisteva un solo “archivio” centrale e se quello andava giù, si perdeva tutto.



Git funziona da **riga di comando** (il terminale), ma esistono anche interfacce grafiche (come GitHub Desktop, GitKraken, o l'integrazione diretta in Visual Studio Code) che rendono tutto più visuale. In questo corso vedrai comandi da terminale a scopo illustrativo: non devi memorizzarli a memoria, l'obiettivo è che tu capisca **la logica**.

4.2 Cos'è GitHub (e non è la stessa cosa di Git!)

Nel diagramma di prima abbiamo scritto “GitHub” sopra il server remoto un po' di sfuggita: è il momento di chiarire di cosa si tratta davvero. Uno degli equivoci più comuni per chi inizia, infatti, è pensare che Git e GitHub siano la stessa cosa. Non lo sono.

	Git	GitHub
Cos'è	Uno strumento (software)	Una piattaforma online (un servizio)
Dove vive	Sul tuo computer	Su internet (cloud), o su un server aziendale (GitHub Enterprise)
Cosa fa	Gestisce la cronologia delle modifiche	Ospita i repository online e aggiunge funzionalità di collaborazione
Necessario?	Sì, è il motore	No, è un servizio che <i>usa</i> Git

Analogia: Git è come il motore di un'auto: fa il lavoro tecnico di spostare la macchina. GitHub è come una concessionaria con parcheggio, assistenza e servizi extra: ospita l'auto, la rende visibile ad altri, offre strumenti di manutenzione condivisa. Potresti guidare l'auto (usare Git) senza mai passare dalla concessionaria (senza mai usare GitHub) — ma se lavori in team, avere un posto centrale dove parcheggiare e collaborare è estremamente comodo.

GitHub, in concreto, offre:

- uno spazio online dove “ospitare” i repository (i progetti);
- strumenti di collaborazione come **Pull Request**, **Issue**, revisioni del codice, commenti;
- automazioni (**GitHub Actions**, che vedremo nella sezione CI/CD);
- gestione di permessi, team, sicurezza.

GitHub **non è l'unica piattaforma** di questo tipo: esistono alternative diffuse con funzionalità molto simili, a volte con nomi leggermente diversi per gli stessi concetti — ma la logica di fondo resta sempre identica: Git sotto il cofano, una piattaforma di collaborazione sopra.

Il concetto di fondo è sempre lo stesso (Git sotto il cofano), cambia la piattaforma “concessionaria” sopra. In questo corso useremo GitHub come riferimento perché è la piattaforma usata realmente dal team, sia in versione cloud (github.com) sia, in alcuni contesti aziendali più grandi, in versione **self-hosted** con **GitHub Enterprise Server** (installata sui server dell'azienda, ad esempio su un indirizzo come github.tuaazienda.it), quando l'azienda vuole mantenere il controllo completo della propria infrastruttura.

4.3 Repository: il “progetto” versionato

GitHub ospita i progetti, dicevamo: ma cosa contiene esattamente uno di quei progetti da un punto di vista tecnico? La risposta è il repository. Un **repository** (spesso abbreviato “repo”) è semplicemente **la cartella di un progetto**, ma tracciata da Git. Contiene tutti i file del progetto (codice, documentazione, configurazioni) più una cartella speciale invisibile chiamata **.git**, dove Git conserva tutta la cronologia delle modifiche.

Analogia: se il progetto è un libro, il repository è sia il libro stesso sia l'archivio completo di tutte le bozze precedenti, capitolo per capitolo, con la data e l'autore di ogni modifica.

Un repository può essere: - **locale**: la copia che vive sul tuo computer; - **remoto**: la copia ospitata online (su GitHub, cloud o self-hosted), che funge da punto di incontro condiviso tra tutti i membri del team.

```
# Creare un nuovo repository da zero, dentro una cartella
git init

# Oppure "clonare" (scaricare) un repository che esiste già online
git clone https://github.com/nome-organizzazione/nome-progetto.git
```

Dopo un `git clone`, avrai sul tuo computer una copia completa del progetto, compresa tutta la sua storia — pronta per essere modificata.

Esempio pratico: immagina che Marco ti mandi il link al repository GitHub del progetto, ad esempio `https://github.com/shopfacile/backend-api` (o, se l'azienda usa un'istanza self-hosted, qualcosa come `https://github.tuaazienda.it/shopfacile/backend-api`). Aprendolo nel browser vedrai: una lista di cartelle e file (il codice), un pulsante verde “Code” per clonarlo, il numero di branch attivi, e una voce “Commits” con la cronologia di tutte le modifiche fatte finora. Se lo clonassi sul tuo computer con `git clone https://github.com/shopfacile/backend-api.git`, ti ritroveresti in una cartella `backend-api` con dentro tutti quei file, pronti per essere letti — anche se non li modifichi mai.

Maven e il file POM: gestire le librerie che il team non ha scritto

Un repository come `backend-api`, però, non contiene *tutto* il codice che il software usa davvero. Il software di oggi non è scritto tutto in casa: un'applicazione tipica si appoggia a decine di librerie scritte da altri (per parlare con un database, per gestire le date, per validare un'email), e ognuna di quelle librerie usa a sua volta altre librerie. Scaricarle a mano una per una e infilarle nel repository è ingestibile — e il classico “sul mio computer funziona” nasce spesso proprio da due sviluppatori con due versioni leggermente diverse della stessa libreria installate a mano.

Nel mondo Java, lo strumento che risolve questo problema si chiama **Maven**: gestisce sia la build del progetto (compilazione, test, creazione del pacchetto finale) sia le sue dipendenze esterne. Il cuore di Maven è un file chiamato **POM (Project Object Model)**, quasi sempre salvato come `pom.xml` nella radice del repository — è lì che, aprendo un progetto Java, riconoscerai a colpo d'occhio “questo usa Maven”.

```

<project>
  <groupId>it.shopfacile</groupId>
  <artifactId>backend-api</artifactId>
  <version>2.4.0</version>          <!-- versione del progetto (sezione 3) -->

  <dependencies>
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <version>42.7.3</version>      <!-- una dipendenza dichiarata, con versione
        precisa -->
    </dependency>
  </dependencies>
</project>

```

Poche cose contano davvero, per riconoscere e dialogare, non per scrivere XML: la dipendenza dichiarata nel `pom.xml` porta con sé anche le **dipendenze transitive**, quelle di cui nessuno nel team ha scritto una riga né chiesto esplicitamente — arrivano perché la libreria che hai dichiarato ne usa altre a sua volta — e sono spesso il punto in cui compaiono le vulnerabilità note di sicurezza (approfondito nella sezione 13). In azienda, queste librerie non si scaricano da internet a ogni build: vengono scaricate una volta e conservate in un **repository di artifact** interno (strumenti come Nexus o Artifactory), sia per velocità sia per non dipendere da internet in produzione. Il ciclo tipico di build Maven è `compile → test → package → install → deploy`, lo stesso spirito delle fasi di pipeline che vedrai nella sezione 10. Ecosistemi diversi risolvono lo stesso problema con nomi diversi: **npm** per JavaScript (già citato nella sezione 10), **pip** per Python, **NuGet** per .NET — stesso ruolo di Maven, in un altro mondo.

4.4 Commit: uno scatto fotografico delle modifiche

Il repository conserva “tutta la cronologia delle modifiche”, abbiamo detto: ma di cosa è fatta, concretamente, questa cronologia? È fatta di commit. Un **commit** è un salvataggio “ufficiale” di un insieme di modifiche, con un messaggio che spiega **cosa** e **perché** è stato cambiato.

Analogia: immagina di scattare una fotografia allo stato attuale del progetto. Ogni commit è uno scatto: cattura esattamente come erano i file in quel momento, con un’etichetta (il messaggio di commit) che descrive cosa è successo da uno scatto all’altro. Con tante fotografie in sequenza, hai un album completo della storia del progetto.

Il flusso tipico per creare un commit è:

```
# 1. Vedo quali file ho modificato
git status

# 2. "Metto in valigia" le modifiche che voglio salvare (staging)
git add nome-file.txt
# oppure, per aggiungere tutti i file modificati:
git add .

# 3. Creo il commit con un messaggio descrittivo
git commit -m "Aggiunge la validazione del campo email nel form di registrazione"
```

Alcune buone pratiche sui messaggi di commit (le vedrai spesso menzionate nelle Pull Request del team):

- il messaggio deve spiegare **il perché**, non solo il “cosa” (il “cosa” si vede già dal codice cambiato);
- meglio tanti commit piccoli e mirati che un unico commit enorme con scritto “fix vari”;
- molti team adottano convenzioni standard, ad esempio i [Conventional Commits](#) (**feat:** , **fix:** , **docs:** , **chore:** ...).

Ogni commit ha un **identificativo univoco** (un codice esadecimale, es. **a1b2c3d**), che permette di riferirsi esattamente a quello scatto in qualsiasi momento futuro — utile per capire, ad esempio, “in quale commit è stato introdotto questo bug?”.

Esempio pratico: dopo tre commit di Marco su ShopFacile, se lanci **git log --oneline** vedresti qualcosa così:

```
a1b2c3d Aggiunge la validazione del campo email nel form di registrazione
9f8e7d6 Aggiunge il form di registrazione utente
3c4d5e6 Setup iniziale del progetto
```

Ogni riga è un commit: il codice a sinistra (es. **a1b2c3d**) è l'identificativo univoco, il testo a destra è il messaggio. Se un giorno Giulia nota un bug nella validazione email, potrà usare **git log** per capire esattamente in quale commit — e quindi in che momento — è stata introdotta quella modifica.

La cronologia dei commit è per sempre: qualsiasi cosa finisca in un commit resta nella storia del repository per sempre, anche se in un commit successivo cancelli quel file. Chiunque abbia accesso al repository può tornare indietro fino al primo commit e recuperarla.

La conseguenza operativa, importante anche per un PM: se una password, una chiave API o un altro segreto finisce per errore in un commit, **non basta rimuovere il file**. Quella credenziale va considerata compromessa e va **sostituita subito**, ovunque venga usata — riscrivere la cronologia per “far finta che non sia successo” è un’operazione tecnica complessa e rischiosa, non una soluzione al problema.

Lo strumento che previene tutto questo a monte è il file `.gitignore`: elenca quali file (chiavi, credenziali, file temporanei, cartelle di build) Git deve ignorare, evitando che entrino per errore in un commit.

4.5 Branch: un ramo di lavoro parallelo

Ogni commit di Marco si accumula in sequenza sulla stessa linea temporale: ma cosa succede quando Giulia e Ahmed devono lavorare *contemporaneamente* su cose diverse, senza pestarsi i piedi? Serve poter deviare dalla linea principale, ed è qui che entra in gioco il branch.

C’è una posta in gioco più seria del semplice “non ostacolarsi”: ShopFacile è già online, con clienti che stanno pagando in questo momento. Se Marco, per aggiungere un nuovo metodo di pagamento, lavorasse direttamente su `main`, un suo errore fermerebbe il checkout subito, per tutti. Il branch è prima di tutto una **rete di sicurezza verso la produzione**, e solo in secondo luogo un modo per non ostacolare i colleghi.

Un **branch** (letteralmente “ramo”) è una linea di sviluppo indipendente che parte da un punto della storia del progetto. Permette di lavorare su qualcosa di nuovo (una funzionalità, una correzione) **senza toccare** la versione “stabile” del progetto, che di solito vive nel branch principale chiamato `main` (in passato spesso chiamato `master`). La transizione da un nome all’altro si è diffusa intorno al 2020: la ragione più citata è la volontà di abbandonare una coppia di termini (`master` / `slave`, usata in informatica per indicare un componente che comanda e uno che obbedisce) letta da molti come inopportuna. Il passaggio non è stato universale, così entrambi i nomi si incontrano ancora nei progetti esistenti: se apri un repository più vecchio e trovi `master`, è semplicemente il branch principale con il vecchio nome.

Analogia: pensa alla linea principale della storia (`main`) come alla strada maestra di un progetto: deve restare sempre percorribile e affidabile. Un branch è come un cantiere laterale in cui provi cose nuove: se qualcosa va storto, il cantiere non blocca la strada principale. Quando il lavoro nel cantiere è pronto e testato, lo “colleghi” alla strada maestra.

```
# Creo un nuovo branch chiamato "feature/login-utente"
git branch feature/login-utente

# Mi sposto su quel branch per iniziare a lavorarci
git checkout feature/login-utente

# In alternativa, questi due comandi si possono unire in uno solo:
git checkout -b feature/login-utente
```

Convenzioni di naming comuni per i branch (utili da riconoscere quando le vedi nelle Pull Request):

- `feature/...` → nuova funzionalità (es. `feature/login-utente`)
- `fix/...` o `bugfix/...` → correzione di un bug
- `hotfix/...` → correzione urgente su produzione
- `release/...` → preparazione di una nuova versione

Lavorare su branch separati è ciò che permette a più persone del team di lavorare **in parallelo sullo stesso progetto** senza pestarsi i piedi.

Esempio pratico: il team di ShopFacile deve sviluppare due cose in parallelo: una nuova pagina di login e la correzione di un bug nel checkout. Marco e Ahmed lanceranno:

```
git checkout -b feature/login-utente      # Marco
git checkout -b fix/checkout-prezzo-errato # Ahmed
```

Da questo momento lavorano su due “cantieri” separati: i commit di Marco non toccano i file su cui lavora Ahmed (a meno che non modifichino esattamente gli stessi file), e nessuno dei due tocca `main`, che resta stabile per tutti gli altri.

4.6 Merge: unire due rami

Marco e Ahmed, però, non possono lavorare per sempre su rami separati: prima o poi il loro lavoro deve tornare a far parte di un'unica versione condivisa. L'operazione che lo rende possibile è il merge. Il **merge** è l'operazione che **unisce le modifiche** fatte su un branch dentro un altro branch (tipicamente, si uniscono le modifiche di un branch di feature dentro `main`, una volta che il lavoro è completo e testato).

Analogia: riprendendo l'immagine di prima, il merge è il momento in cui il cantiere laterale viene collegato alla strada principale: da quel momento, chi percorre la strada maestra passa anche dal nuovo tratto.


```
# Mi sposto sul branch che deve "ricevere" le modifiche
git checkout main

# Unisco le modifiche del branch di feature dentro main
git merge feature/login-utente
```

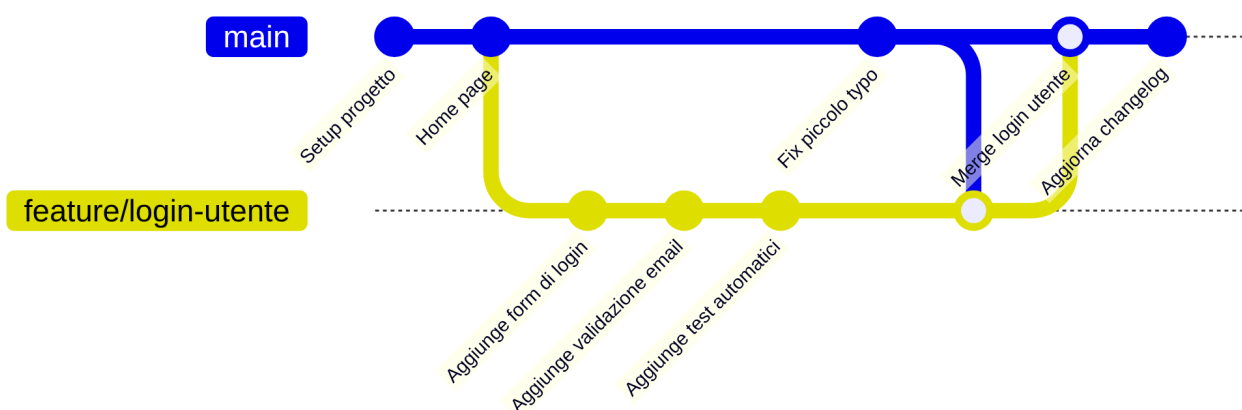
Git è molto bravo a unire automaticamente modifiche fatte su parti diverse dei file. Ma se due persone hanno modificato **esattamente la stessa riga** in modo diverso, Git non sa quale versione tenere: si genera un **conflitto di merge**, che va risolto manualmente decidendo quale modifica (o quale combinazione) mantenere. Non preoccuparti: è un evento normalissimo nel lavoro quotidiano, non un errore grave.

Esempio pratico: supponiamo che il branch `feature/login-utente` sia pronto. Da `main` lanci:

```
git checkout main
git merge feature/login-utente
```

Se nessuno ha toccato le stesse righe su `main` nel frattempo, Git risponde con qualcosa come `Fast-forward` o `Merge made by the 'ort' strategy` e il lavoro è unito automaticamente. Se invece Giulia aveva modificato lo stesso file nello stesso punto, Git si ferma e restituisce un messaggio come `CONFLICT (content): Merge conflict in login.js` — a quel punto tocca aprire il file, decidere quale versione tenere (o combinarle), e completare il merge con `git add + git commit`.

Ecco un diagramma che riassume l'intero ciclo di vita di un branch di feature, dalla creazione al merge:



Cosa racconta questo diagramma: mentre il branch `feature/login-utente` procedeva con i suoi commit, anche `main` è avanzato con un piccolo fix. Il branch di feature ha “assorbito” quell’aggiornamento (merge di `main` dentro il branch), per essere sicuro di lavorare sulla versione più recente, e infine è stato unito (merge) dentro `main`, portando tutto il lavoro fatto nella linea principale.

4.7 Pull Request: il cuore della collaborazione su GitHub

Abbiamo visto come funziona un merge da riga di comando, ma nella pratica quotidiana su GitHub il merge non viene quasi mai lanciato “a freddo”: passa prima per uno strumento che formalizza la richiesta e apre lo spazio per la revisione, la Pull Request. Una **Pull Request** (spesso abbreviata **PR**) è una **richiesta formale** di unire le modifiche di un branch dentro un altro branch (di solito dentro `main`), passando prima per una **revisione** da parte di altri membri del team.

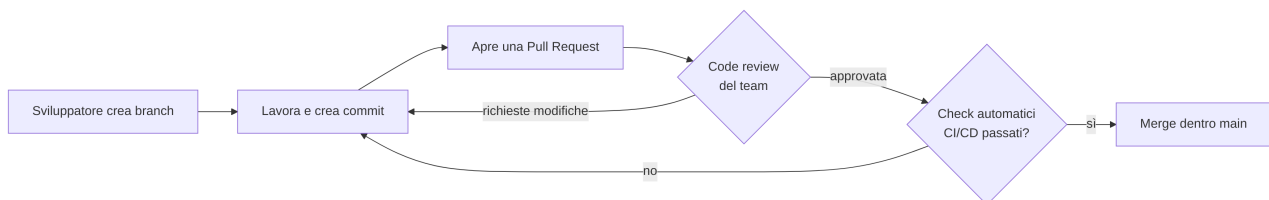
Il nome, tradotto alla lettera, è “richiesta di *tirare dentro*” (pull) il proprio lavoro nel ramo principale: eredità dell’epoca in cui si chiedeva a chi manteneva un progetto open source di prelevare le modifiche da un’altra copia del repository, su un altro computer.

Questo è probabilmente il concetto più importante di tutta la sezione, perché è il meccanismo attraverso cui lavora quasi ogni team che usa GitHub. (Su altre piattaforme lo stesso identico concetto può avere un nome diverso — non farti confondere dal nome: la logica è la stessa.)

Analogia: è come sottoporre una bozza di documento a un collega prima di renderla definitiva. Non modifichi direttamente il documento ufficiale: proponi le tue modifiche, il collega le legge, lascia commenti, magari chiede correzioni, e solo quando tutti sono d’accordo la bozza diventa la versione ufficiale.

Il flusso tipico di una Pull Request:

1. Uno sviluppatore (ad esempio Marco) crea un branch e ci lavora (con i suoi commit).
2. Quando il lavoro è pronto, apre una **Pull Request** su GitHub, proponendo di unire quel branch dentro `main` (o dentro un altro branch di destinazione).
3. Altri membri del team fanno la **code review**: leggono le modifiche, lasciano commenti, chiedono chiarimenti o correzioni.
4. Spesso, in parallelo, delle **pipeline automatiche** (CI/CD, che vedremo nella sezione 10) eseguono automaticamente test e controlli di qualità sul codice della PR.
5. Quando tutto è approvato (“approved”) e i controlli automatici sono passati (i cosiddetti “check verdi”), la Pull Request viene unita (merge) — spesso con un semplice click sul pulsante “Merge” su GitHub.



Perché le Pull Request sono così importanti per un Project Manager? Perché sono il punto in cui puoi **misurare visivamente lo stato di avanzamento** del lavoro: quante PR sono aperte, quante sono in attesa di revisione, quanto tempo restano aperte prima di essere unite (un indicatore utile di quanto è fluido il processo del team).

Esempio pratico: Marco ha finito la feature login su ShopFacile. I passaggi concreti su GitHub sarebbero: 1. Push del branch: `git push origin feature/login-utente`. 2. Su GitHub compare un banner “Compare & pull request”: clicca, scrivi un titolo (es. “Aggiunge login utente con validazione email”) e una descrizione di cosa cambia e perché. 3. Assegna Giulia come reviewer. 4. Giulia lascia commenti tipo “Perché qui usiamo `==` invece di `===`?” — Marco risponde o corregge con un nuovo commit sullo stesso branch, che aggiorna automaticamente la PR. 5. Quando i commenti sono risolti e i check automatici (CI/CD) sono verdi, Giulia clicca “Approve”, poi qualcuno clicca “Merge pull request”.

Da Project Manager, vedresti questa PR nella sezione “Pull requests” del repository con un’etichetta verde “Open” che diventa viola “Merged” a fine processo.

4.8 Issue: tracciare bug e richieste

Finora abbiamo parlato di codice già scritto: branch, commit, pull request. Ma da dove nasce il lavoro stesso, cioè “cosa” bisogna scrivere o correggere?

Prima che ShopFacile adottasse le Issue, i bug segnalati dai clienti finivano in email sparse o in un foglio Excel condiviso aggiornato a intermittenza: Sara si ritrovava a chiedere in chat “qualcuno si sta già occupando del bug del checkout?” senza risposta certa, perché nessuno sapeva se un problema fosse già noto, già assegnato o già risolto.

Spesso il lavoro nasce da una Issue. Una **Issue** è una “voce” che descrive un problema da risolvere o una richiesta da realizzare: un bug da correggere, una nuova funzionalità da sviluppare, un miglioramento da valutare.

Analogia: è come un “biglietto” (ticket) di un help desk: qualcuno segnala un problema o una richiesta, il biglietto viene assegnato a qualcuno, discusso, e chiuso quando risolto.

Una Issue tipicamente contiene: - un titolo chiaro; - una descrizione del problema o della richiesta (spesso con passi per riprodurre un bug, screenshot, comportamento atteso vs. osservato); - **etichette** (label), es. `bug`, `enhancement`, `documentazione`; - un **assegnatario** (chi ci lavora); - commenti di discussione.

Le Issue si possono collegare direttamente alle Pull Request: è comune vedere in una PR la frase “Closes #42”, che indica che, una volta unita quella PR, la Issue numero 42 verrà chiusa automaticamente perché risolta.

Molti team di progetto (incluso probabilmente il tuo) usano le **Issue di GitHub** come base per la gestione del backlog che vedremo nelle sezioni su Agile, Scrum e Kanban.

Esempio pratico: un cliente di ShopFacile segnala che il pulsante “Conferma ordine” non funziona su un certo browser. Sara apre una Issue così: - **Titolo:** “Il pulsante Conferma ordine non risponde su Safari” - **Descrizione:** passi per riprodurlo, screenshot, browser e versione usati - **Label:** `bug`, `priorità-alta` - **Assegnatario:** Ahmed, che se ne occuperà

Ahmed apre un branch `fix/conferma-ordine-safari`, risolve il problema, e nella descrizione della Pull Request scrive “Closes #57” (dove 57 è il numero della Issue): una volta unita la PR, GitHub chiude automaticamente la Issue numero 57.

4.9 Release: una versione pubblicata ufficialmente

La Issue #57 è stata chiusa, il fix è unito in `main`: ma quando quel fix arriva davvero nelle mani degli utenti, con quale nome viene comunicato? Con quello di una Release. Una **Release** è un punto preciso della storia del progetto che viene “pubblicato” ufficialmente come versione consegnabile agli utenti finali (o al cliente).

Analogia: se i commit sono gli scatti fotografici quotidiani di un album, la release è la **foto scelta per la copertina** di un capitolo: un momento speciale, marcato e comunicato a tutti come “questa è la versione 2.3.0, disponibile da oggi”.

Le release seguono spesso uno schema di numerazione chiamato **Semantic Versioning** (`MAJOR.MINOR.PATCH`, es. `2.3.1`): - **MAJOR:** cambiamenti grandi, potenzialmente non compatibili con le versioni precedenti (es. `1.x` → `2.0.0`); - **MINOR:** nuove funzionalità compatibili con le versioni precedenti (es. `2.2.x` → `2.3.0`); - **PATCH:** correzioni di bug, senza nuove funzionalità (es. `2.3.0` → `2.3.1`).

Su GitHub, una Release viene solitamente accompagnata da **note di rilascio** (release notes): un elenco leggibile di cosa è cambiato, utilissimo anche per un Project Manager che deve comunicare al cliente o agli stakeholder cosa contiene la nuova versione.

Esempio pratico: il team di ShopFacile rilascia la versione `2.3.0` del prodotto. Sulla pagina “Releases” di GitHub troveresti una voce con: - tag `v2.3.0`; - titolo “Versione 2.3.0 — Gestione utenti”; - note di rilascio tipo: “Nuove funzionalità: gestione ruoli utente. Fix: corretto un bug nel checkout. Nota: richiede un aggiornamento del database.”; - eventuali file scaricabili (es. un installer).

Un Project Manager potrebbe riprendere direttamente queste note di rilascio — magari semplificandole — in una comunicazione al cliente o in un aggiornamento agli stakeholder.

4.10 Tag: etichettare un punto preciso della storia

Ogni Release che abbiamo appena visto, in realtà, si appoggia su un meccanismo più elementare di Git per individuare quel punto preciso della storia: il tag. Un **Tag** è un'etichetta che punta a un commit specifico, per marcarlo come "punto di interesse" — quasi sempre usato insieme alle release.

Analogia: se la storia del progetto è un lungo rotolo di pellicola fotografica, un tag è un segnalibro adesivo attaccato a uno scatto preciso, con scritto sopra "v2.3.0" — così puoi ritrovarlo istantaneamente anche tra migliaia di altri scatti.

```
# Creo un tag "annotato" (con messaggio) sul commit corrente
git tag -a v2.3.0 -m "Versione 2.3.0: aggiunta gestione utenti"

# Invio il tag anche al repository remoto (altrimenti resta solo locale)
git push origin v2.3.0
```

Differenza pratica tra Tag e Release: il **tag** è un concetto di Git (un puntatore a un commit), mentre la **Release** è un concetto di GitHub costruito sopra un tag, con in più note descrittive, file scaricabili (binari, installer) e visibilità nella pagina del progetto.

Esempio pratico: dopo aver taggato e pubblicato la versione **v2.3.0**, se sei sul repository e lanci **git tag**, vedresti l'elenco di tutti i tag creati nel tempo:

```
v1.0.0
v1.1.0
v2.0.0
v2.1.0
v2.2.0
v2.3.0
```

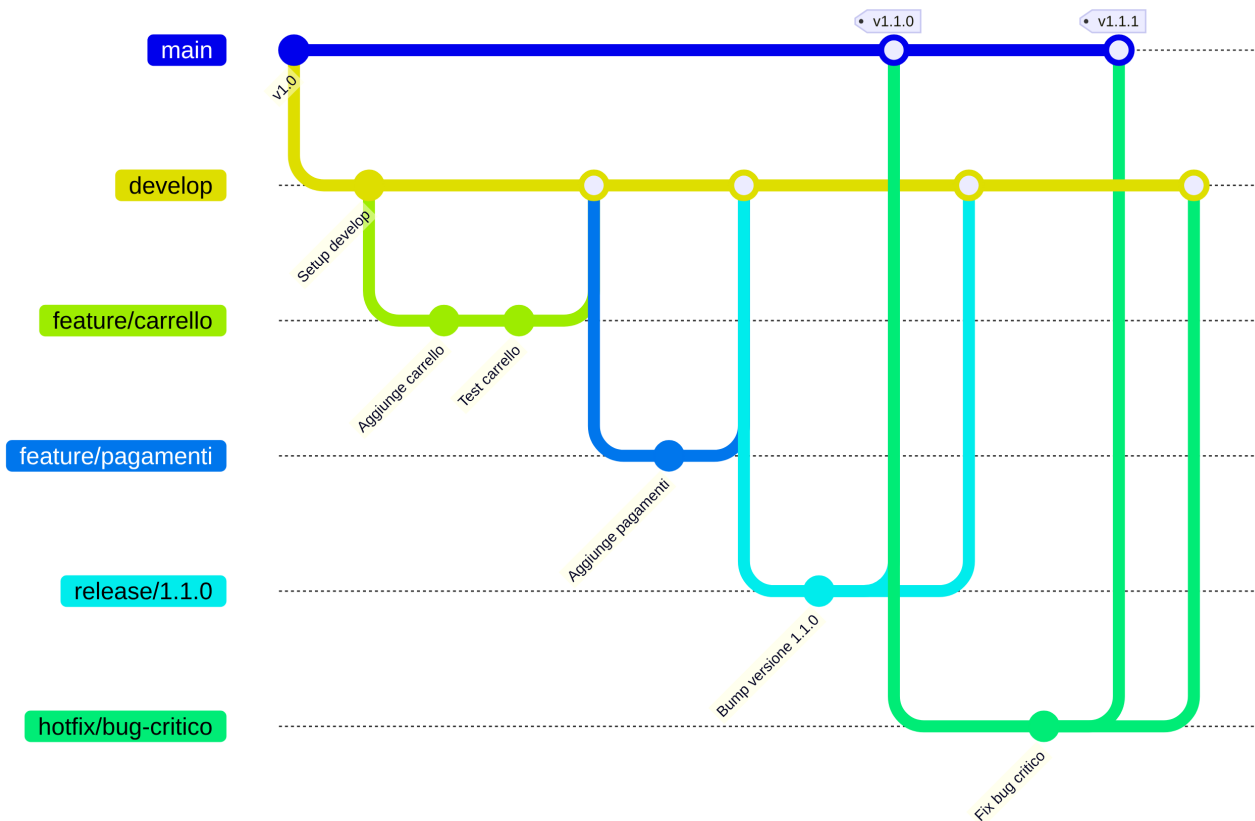
Se un giorno serve tornare esattamente alla versione rilasciata qualche mese prima, basta fare **git checkout v2.1.0** (se quello era il tag di quella release) per vedere il progetto esattamente come era in quel momento, senza dover cercare "a occhio" nella cronologia dei commit.

4.11 Git Flow: un workflow strutturato a più branch

Finora abbiamo visto i singoli “mattoncini” (branch, merge, PR, tag, release): ma come si combinano insieme in una strategia coerente, giorno per giorno, per tutto il team? Un primo modello molto diffuso è Git Flow. **Git Flow** è un modello di organizzazione dei branch molto diffuso, pensato per progetti con **cicli di rilascio pianificati** (es. una nuova versione ogni mese) e la necessità di gestire più cose in parallelo: nuove funzionalità, preparazione di una release, correzioni urgenti su produzione.

Git Flow prevede branch con ruoli specifici:

- **main** : contiene sempre il codice in produzione, stabile al 100%.
- **develop** : il branch di “integrazione”, dove confluiscono le funzionalità completate, in attesa della prossima release.
- **feature/...** : branch temporanei per sviluppare singole funzionalità, che partono da **develop** e ci ritornano a lavoro finito.
- **release/...** : branch temporanei usati per finalizzare una versione (test, piccoli fix, aggiornamento numero di versione) prima di pubblicarla.
- **hotfix/...** : branch temporanei per correzioni urgenti direttamente su **main**, per problemi critici in produzione che non possono aspettare il prossimo ciclo di release.



Quando usare Git Flow: è utile quando il progetto ha rilasci pianificati e non continui (es. release mensili), quando serve mantenere in parallelo più versioni in produzione, o quando il processo di approvazione/rilascio è complesso e richiede una fase di stabilizzazione prima di ogni release. Lo svantaggio è che è un processo più pesante, con più branch da gestire e più passaggi di merge.

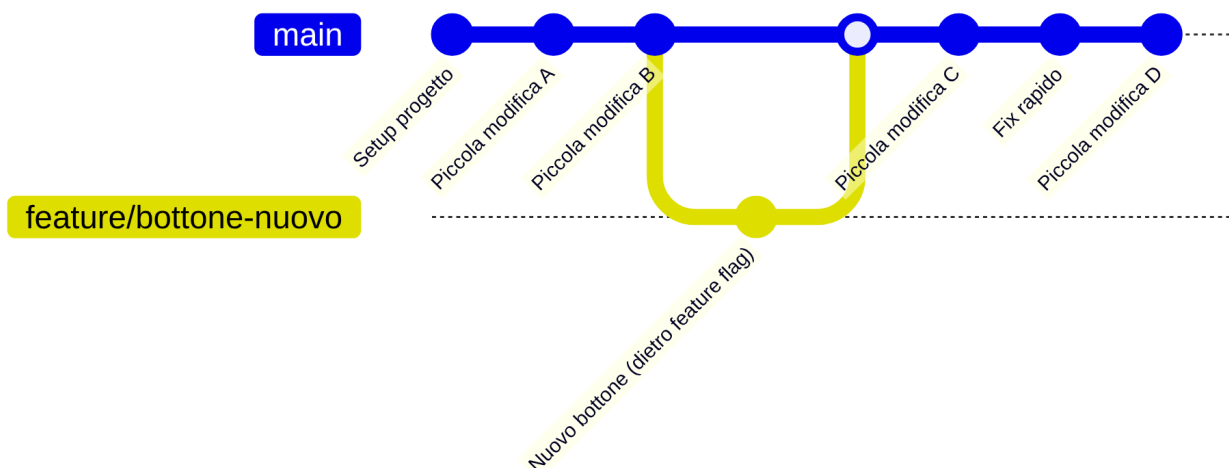
Esempio pratico: immaginiamo che ShopFacile rilasci una nuova versione ogni mese. Lo scenario tipico con Git Flow: - Marco, Giulia e Ahmed lavorano tutto il mese su vari branch `feature/...` che partono da `develop` e ci tornano quando pronti; - una settimana prima del rilascio si crea `release/1.4.0` da `develop`: qui si fanno solo piccoli fix e test finali, niente nuove funzionalità; - quando è tutto stabile, `release/1.4.0` viene unito sia in `main` (con tag `v1.4.0`, che diventa la produzione) sia di nuovo in `develop`; - se due giorni dopo il rilascio emerge un bug critico in produzione, si crea `hotfix/bug-pagamenti` direttamente da `main`, si corregge, si unisce in `main` (nuovo tag `v1.4.1`) e anche in `develop`, così il fix non si perde nel prossimo rilascio.

4.12 Trunk Based Development: il workflow semplificato

Git Flow, con i suoi cinque tipi di branch, funziona bene per rilasci pianificati, ma è un processo pesante per chi rilascia molto più spesso: esiste un'alternativa decisamente più snella. Il **Trunk Based Development** (TBD) è un approccio molto più semplice: tutti i membri del team lavorano il più possibile direttamente su **un unico branch principale** (il "trunk", cioè `main`), con **commit frequenti e piccoli**, integrando il proprio lavoro più volte al giorno.

Analogia: se Git Flow è come costruire un edificio con tanti cantieri separati che vengono collegati alla struttura principale solo a lavori ultimati, il Trunk Based Development è come aggiungere un mattone alla volta direttamente sulla struttura principale, controllando ogni singolo mattone prima di posarlo, così l'edificio resta sempre in piedi e visitabile.

I branch di feature, quando si usano, sono **molto brevi** (durano ore o al massimo un paio di giorni, non settimane), e le funzionalità non ancora pronte per gli utenti finali vengono spesso "nascoste" tramite **feature flag** (interruttori che attivano/disattivano una funzionalità) invece che tenute isolate su un branch a lungo termine.



Quando usare Trunk Based Development: funziona molto bene quando il team ha una **suite di test automatici solida** e una pipeline di CI/CD matura, capace di verificare rapidamente che ogni piccolo cambiamento non rompa nulla. È il modello preferito dai team che fanno rilasci **continui** (più volte al giorno o alla settimana), tipico della cultura DevOps che vedremo più avanti nel corso. Lo svantaggio è che richiede molta disciplina e automazione: senza test solidi, integrare spesso su un unico branch diventa rischioso.

Esempio pratico: immaginiamo che il team di ShopFacile lavori con Trunk Based Development e una pipeline CI/CD solida. Giulia deve aggiungere un nuovo filtro di ricerca: - crea un branch `feature/filtro-ricerca` la mattina; - fa 2-3 commit piccoli nel corso della giornata, ciascuno verificato automaticamente dalla pipeline; - il filtro non è ancora pronto per tutti gli utenti, quindi lo nasconde dietro un feature flag chiamato `nuovo_filtro_ricerca_attivo` (inizialmente spento); - entro sera apre una PR piccola e veloce da revisionare, e la unisce in `main`; - il codice è ora in produzione ma invisibile agli utenti (flag spento); quando il team è pronto, attiva il flag per tutti (o per un gruppo di test) senza dover rilasciare nulla di nuovo.

Confronto rapido

	Git Flow	Trunk Based Development
Struttura branch	Molti branch a lungo termine (<code>main</code> , <code>develop</code> , <code>feature</code> , <code>release</code> , <code>hotfix</code>)	Un solo branch principale, branch di feature brevissimi
Frequenza integrazione	Bassa (le feature restano isolate a lungo)	Alta (più commit al giorno su <code>main</code>)
Adatto a	Rilasci pianificati, versioni multiple in produzione	Rilasci continui, cultura DevOps, forte automazione
Serve	Processo chiaro e disciplina sui merge	Test automatici solidi, feature flag
Rischio principale	Conflitti di merge grandi e complessi	Rischio di “rompere” <code>main</code> se manca automazione

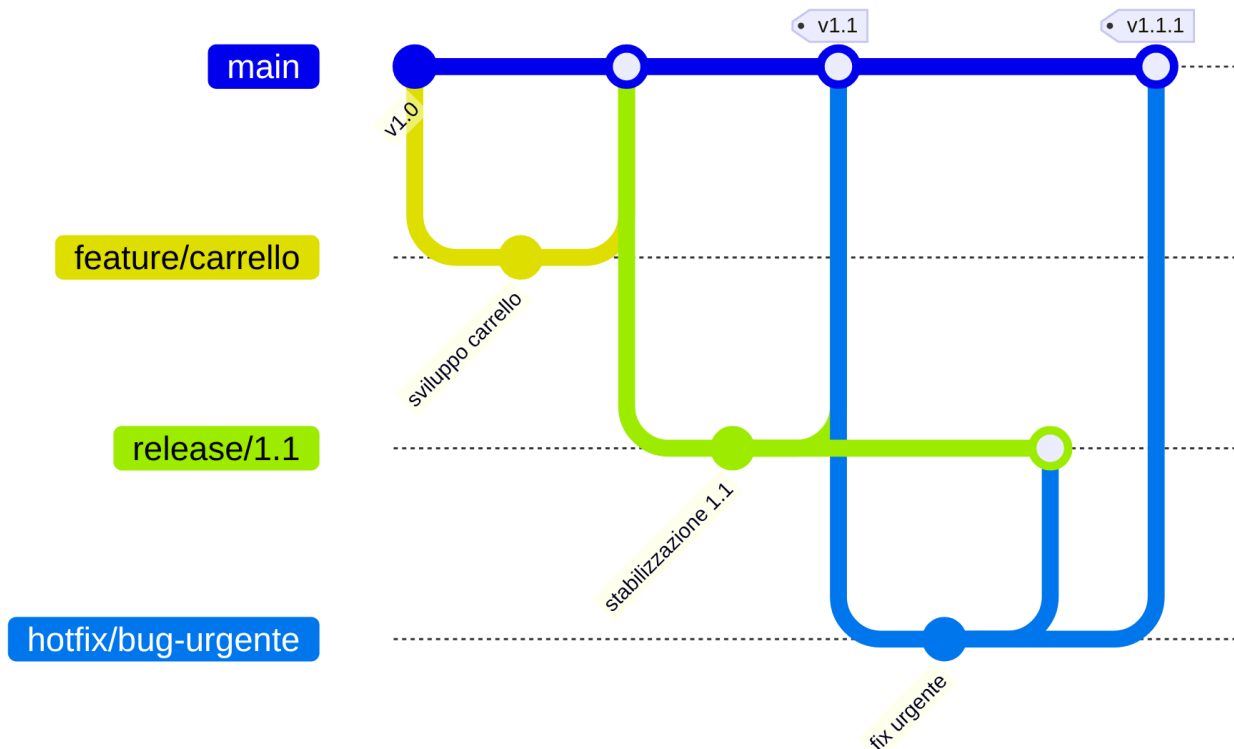
Non esiste un modello “migliore in assoluto”: la scelta dipende dalla maturità del team, dal tipo di prodotto e dalla frequenza di rilascio desiderata. Molti team reali usano versioni semplificate o ibride di questi due modelli, adattandole al proprio contesto — e la via di mezzo più diffusa ha un nome proprio, che vale la pena conoscere prima di incontrarlo in una riunione.

4.13 Il modello “cactus” (OneFlow): una via di mezzo

Git Flow, con `main` e `develop` sempre vivi in parallelo, raddoppia i punti in cui il codice può divergere: ogni feature branch deve tenersi allineato con `develop`, e `develop` a sua volta con `main` a ogni release. Trunk Based Development risolve il problema riducendo tutto a un solo branch di lunga

vita, ma **richiede** una suite di test automatici molto solida e feature flag maturi — condizioni che non tutti i team hanno il primo giorno. Nella pratica, molte aziende non stanno né perfettamente nell'uno né nell'altro: stanno in mezzo.

Il modello che descrive questa via di mezzo è conosciuto informalmente come **modello “cactus”**, e anche come **OneFlow**. L'idea: un solo tronco (**main**) da cui partono ed a cui tornano rami corti di feature — come in Trunk Based — ma da cui, quando serve preparare o mantenere una release, si staccano anche rami di release più longevi, che a loro volta si biforcano solo per le correzioni urgenti (hotfix). Visto nel suo insieme, con il tronco centrale e questi rami corti che si irradiano ai lati, la forma ricorda vagamente un cactus — da cui il nome.



A differenza di Git Flow, però, non esiste un branch **develop** permanente: le feature vanno dirette su **main** (o quasi), e i rami di release sono temporanei, legati a una specifica versione da mantenere, non un secondo tronco parallelo da tenere sincronizzato per sempre.

Un avvertimento pratico più importante del nome stesso: a differenza di “Git Flow”, la terminologia del modello cactus **non è standardizzata**. Team diversi, quando dicono “usiamo un modello cactus” (o “OneFlow”), possono intendere varianti anche sensibilmente diverse tra loro. Se un collega te lo dice in una riunione, la cosa giusta da fare non è annuire come se il nome bastasse a spiegare tutto: è chiedere di disegnarlo sulla lavagna, per capire davvero quali branch vivono a lungo e quali sono temporanei nel progetto specifico su cui lavori.

	Git Flow	Cactus (OneFlow)	Trunk Based
Branch di lunga vita	Due (<code>main</code> , <code>develop</code>)	Uno (<code>main</code>), più i rami di release finché sono mantenuti	Uno solo (<code>main</code>)
Frequenza di rilascio	Pianificata, es. mensile	Flessibile: rilasci frequenti dal tronco, release più stabili quando serve	Continua, più volte al giorno
Correzioni urgenti	Branch <code>hotfix/...</code> da <code>main</code> , poi riportato anche in <code>develop</code>	Branch <code>hotfix/...</code> dal ramo di release interessato, poi riportato su <code>main</code>	Fix diretto su <code>main</code> , spesso dietro feature flag
Requisiti di test automatici	Moderati: la fase di stabilizzazione del branch <code>release/...</code> compensa test meno maturi	Medio-alti: servono buoni test, ma il ramo di release dà un margine di sicurezza in più	Molto alti: senza test solidi, ogni commit su <code>main</code> è un rischio
Contesto in cui conviene	Rilasci pianificati, più versioni in produzione contemporaneamente	Prodotto con rilasci abbastanza frequenti ma che deve anche mantenere versioni precedenti (es. per clienti enterprise)	Prodotto SaaS con un'unica versione in produzione, cultura DevOps matura

4.14 Riepilogo dei comandi visti in questa sezione

Abbiamo visto molti comandi sparsi nei vari esempi di questa sezione: raccogliamoli qui in un unico prontuario, utile come riferimento veloce quando ti troverai a seguire una conversazione tecnica del team.

Comando	Cosa fa
<code>git init</code>	crea un nuovo repository
<code>git clone <url></code>	scarica un repository esistente
<code>git status</code>	mostra i file modificati
<code>git add <file></code>	prepara le modifiche per il commit
<code>git commit -m "messaggio"</code>	salva uno "scatto" delle modifiche
<code>git branch <nome></code>	crea un nuovo branch
<code>git checkout <nome></code>	mi sposto su un branch
<code>git checkout -b <nome></code>	crea e mi sposto su un nuovo branch
<code>git merge <branch></code>	unisce un branch in quello corrente
<code>git tag -a <nome> -m "messaggio"</code>	crea un tag annotato
<code>git push origin <branch\ tag></code>	invia branch/tag al repository remoto

Ciò che conta, nel tuo ruolo, è capire **cosa rappresentano concettualmente** (uno scatto, un ramo, un'unione) per poter seguire con consapevolezza le conversazioni tecniche del team, leggere lo stato di una Pull Request, e capire perché il team ha scelto un certo workflow.

Esercizi pratici

Gli esercizi che seguono ti servono a consolidare i concetti visti in questa sezione con le mani sulla tastiera. Non serve nessun progetto reale: puoi farli tutti con un account GitHub gratuito e un repository di prova che poi puoi anche eliminare.

1. **Crea un repository e fai il tuo primo commit.** Crea un repository di prova su GitHub (pubblico o privato, non importa), clonalo sul tuo computer con `git clone`, crea un file `note.txt` con dentro una riga di testo a piacere, poi lancia `git add note.txt` e `git commit -m "Primo commit di prova"`. **Come verificare:** lanciando `git log`, deve apparire il tuo commit con il messaggio che hai scritto e un identificativo esadecimale univoco.
2. **Crea un branch, modificalo e uniscilo.** Nel repository di prova, crea un branch con `git checkout -b feature/prova`, modifica `note.txt` aggiungendo una seconda riga, fai un commit, poi torna su `main` con `git checkout main` e lancia `git merge feature/prova`. **Come verificare:** dopo il merge, `note.txt` su `main` deve contenere anche la riga che hai aggiunto sul branch, e `git log` deve mostrare il commit del branch unito nella storia di `main`.
3. **Genera un conflitto di merge e risolvi.** Sempre su `main`, modifica la prima riga di `note.txt` e fai un commit. Poi crea un nuovo branch da un punto *precedente* a quella modifica (o modifica la stessa riga su un secondo branch prima del merge) e prova a fare il merge: dovresti ottenere un `CONFLICT (content)`. Apri il file, scegli quale testo mantenere, rimuovi i

marcatori `<<<<<<`, `=====`, `>>>>>>` che Git inserisce, e completa il merge con `git add note.txt` e `git commit`. **Come verificare:** `git status` non deve più segnalare conflitti, e `note.txt` deve contenere solo il testo finale che hai scelto, senza marcatori residui.

4. **Apri una Pull Request fittizia su GitHub.** Fai il push del branch `feature/prova` (o uno nuovo) con `git push origin feature/prova`, poi su GitHub apri una Pull Request verso `main`, scrivendo un titolo e una breve descrizione di cosa cambia. Se hai un amico o un collega disponibile, chiedigli di lasciarti un commento; altrimenti lasciatelo da solo per vedere come funziona l'interfaccia. **Come verificare:** la Pull Request compare nella sezione "Pull requests" del repository con stato "Open", e mostra correttamente i file modificati nel tab "Files changed".
5. **Crea un tag e una Release.** Sul repository di prova, crea un tag annotato con `git tag -a v0.1.0 -m "Prima versione di prova"`, invialo con `git push origin v0.1.0`, poi su GitHub vai nella sezione "Releases" e pubblica una Release basata su quel tag, con qualche nota di rilascio scritta da te. **Come verificare:** il tag `v0.1.0` compare lanciando `git tag`, e la Release compare nella pagina "Releases" del repository con le note che hai scritto.
6. **Disegna il workflow del tuo progetto reale.** Chiedi a un collega developer quale workflow (Git Flow, Trunk Based Development, modello cactus/OneFlow, o una via di mezzo) usa il progetto su cui sei inserito/a, e provate insieme a disegnare — anche su un foglio — la sequenza di branch che si crea per una feature "tipica" del progetto, dalla creazione del branch fino al rilascio in produzione. **Come verificare:** sai indicare, senza guardare gli appunti, a quale dei tre modelli visti in questa sezione il progetto si avvicina di più (o se è un ibrido), e sai spiegare perché quella scelta si adatta al ritmo di rilascio del progetto.
7. **Riconosci un `pom.xml`.** Chiedi a un developer del progetto (se il backend è Java) di farti vedere il file `pom.xml` reale, oppure cercane uno di esempio online. Individua senza aiuto: il nome e la versione del progetto, almeno una dipendenza dichiarata con la sua versione. **Come verificare:** sai indicare a un collega, guardando il file, quale libreria esterna il progetto usa e quale versione — senza dover capire una riga di XML in più del necessario.

Collegamenti

- [3. Come nasce un software](#) — dove si inserisce Git nel ciclo di vita di un progetto software, e dove trovi gli ADR per documentare le decisioni tecniche
- [5. Agile](#) — il mindset dietro le pratiche di integrazione frequente che abbiamo visto con il Trunk Based Development
- [9. DevOps](#) — la cultura di automazione e rilascio continuo che rende possibile il Trunk Based Development
- [10. CI/CD](#) — le pipeline automatiche (GitHub Actions) che si attivano su commit e Pull Request, e dove il ciclo di build di Maven si collega alla generazione dell'artifact
- [13. Sicurezza](#) — dove si approfondisce come le dipendenze (comprese quelle transitive viste in questa sezione) possono introdurre vulnerabilità note
- [16. Glossario](#) — per ripassare velocemente i termini di questa sezione

Risorse

- [Documentazione ufficiale di Git](#)
- [Git — Book \(Pro Git, gratuito e in italiano\)](#)
- [GitHub Docs](#)
- [Learn Git Branching \(esercizi interattivi nel browser\)](#)
- [GitHub Docs — Get started](#)
- [About pull requests — GitHub Docs](#)
- [Conventional Commits](#)
- [Semantic Versioning](#)
- [Trunk Based Development](#)
- [OneFlow — a Git branching model and workflow](#) — la descrizione originale del modello “cactus”
- [Apache Maven — Guida introduttiva](#)
- [A successful Git branching model \(Git Flow, Vincent Driessen\)](#)